

AI Engineer Assignment – GenAI Banking Support Chatbot

Objective

Build and deploy an AI-powered Banking Support Chatbot capable of answering banking-related customer queries using Retrieval-Augmented Generation (RAG).

This assignment is intended to evaluate:

- Generative AI knowledge
- RAG implementation
- Chatbot architecture
- API/backend development
- Vector database understanding
- Cloud deployment skills
- Production-oriented engineering thinking

Problem Statement

A fintech company wants an intelligent AI assistant to help customers with:

- Loan-related queries
- Credit card information
- Banking FAQs
- Policy/document assistance
- Customer support knowledge retrieval

The chatbot should retrieve information from banking-related documents and generate context-aware responses.

Functional Requirements

1. Conversational Chatbot Interface

Build a chatbot interface supporting:

- Interactive conversation
- Chat history
- Loading/typing indication
- Session-based conversation flow

2. RAG Pipeline (Mandatory)

The chatbot must implement a complete Retrieval-Augmented Generation pipeline including:

- Document ingestion
- Text chunking
- Embedding generation
- Vector database storage
- Semantic retrieval
- Context-aware LLM response generation

3. Vector Database Requirement (Mandatory)

Use any free vector database or vector storage solution.

Examples:

- ChromaDB
- FAISS

The candidate should:

- Store embeddings in the vector DB
- Perform similarity search
- Retrieve top relevant chunks for generation

4. Data Requirement

Candidates may use:

Option A – Publicly Available Data

Examples:

- Banking FAQs
- Loan policy documents
- Credit card terms & conditions
- RBI/public banking documentation

OR

Option B – AI-Generated Synthetic Data

Candidates may generate realistic banking-related documents using AI tools/LLMs.

Examples:

- Mock customer support documents
- Sample loan FAQs
- Credit card policies
- Banking procedure manuals

The dataset should contain enough information to properly test retrieval quality and contextual responses.

5. Supported Documents

Support document ingestion for:

- PDF
- TXT

(Optional: DOCX)

6. Context Retention

The chatbot should maintain conversational context within a session.

Example:

User: What is a personal loan?

User: What is the interest rate for it?

The chatbot should understand that “it” refers to the personal loan discussed earlier.

7. Backend APIs (Mandatory)

Expose backend APIs such as:

POST /chat

POST /upload

GET /health

Cloud Deployment (Mandatory)

The application must be publicly deployed using any free-tier cloud platform.

Examples:

- [AWS Free Tier](#)
- [Google Cloud Free Tier](#)
- [Render](#)
- [Railway](#)

Deliverables

1. GitHub Repository

Include:

- Source code
- README
- Setup instructions
- Architecture explanation

2. Deployment Link

Provide a working deployed URL.

3. Architecture Diagram

Include:

- frontend
- backend

- LLM generation flow

4. Demo Video (5–10 mins)

Explain:

- overall architecture
- RAG flow
- vector DB usage
- deployment approach
- challenges faced
- future improvements

Evaluation Criteria

Area	Weight
RAG Implementation	25%
Vector DB Usage & Retrieval Quality	20%
Cloud Deployment	15%
Code Quality	15%
Chatbot Accuracy & Relevance	15%
API Design	5%
UI/UX	5%

Bonus Features (Optional)

Additional credit for:

- Streaming responses
- Authentication
- Redis caching
- CI/CD
- Prompt optimization

- Conversation memory improvements
- Reranking

Constraints

- Use free-tier/free resources only.
- Do not hardcode credentials or secrets.
- Handle invalid inputs gracefully.
- Responses should prioritize factual and grounded answers.

Timeline

Recommended completion time:

- 3–5 days